Design and Implementation of a DSL Compiler

Compiler Design - 21CSC304J

Balaji Madhan

RA2311026050283

AIML – D

SRM institute of Science & Technology (SRMIST)
Tiruchirappalli, Tamil Nadu, India.

Instructor: Dr. Gunasundari

Submission Date:

1. Introduction

A compiler is a system that translates high-level source code into a lower-level representation. In this project, a simple Domain-Specific Language (DSL) compiler is designed to demonstrate the fundamental phases of compilation such as lexical analysis and syntax analysis.

The goal of this project is to build a basic compiler pipeline using tools like Flex, Bison, and C.

---

2. Architecture of the Compiler

The compiler follows a modular pipeline:

Input → Lexical Analysis → Syntax Analysis → Output

- **Input:** Source code written in DSL
- **Lexical Analysis:** Converts input into tokens
- **Syntax Analysis:** Validates structure using grammar rules
- **Output:** Displays parsing result

---

3. Tools and Technologies Used

- **Flex** – for lexical analysis
- **Bison** – for parsing
- **C Language** – for implementation
- **GitHub** – for version control and project hosting

---

4. Modules Explanation

4.1 Lexical Analyzer (Lexer)

The lexer reads input and converts it into tokens such as:

- Identifiers (ID)
- Numbers (NUMBER)
- Operators (+, *, =)

This simplifies further processing by the parser.

4.2 Syntax Analyzer (Parser)

The parser checks whether the input follows valid grammar rules.

Example grammar:

stmt → ID = expr
expr → expr + term | term
term → term * factor | factor

If the input is valid, it prints a success message.

4.3 Output Module

The system outputs:

Parsed Successfully

indicating that the input expression is syntactically correct.

---

5. Sample Input and Output

Input:

x = 5 + 3

Output:

Parsed Successfully

---

6. Implementation Details

The project is implemented using a structured folder layout:

- src/ → Contains lexer and parser files
- test/ → Sample input files
- output/ → Output results
- docs/ → Documentation

The integration of Flex and Bison enables automatic generation of lexical and parsing components.

---

7. Conclusion

This project demonstrates the basic working of a compiler using a DSL. It highlights how source code is processed through different stages to ensure correctness. The implementation provides a clear understanding of compiler design principles in a simplified manner.